
Pipelined AES-128 Encryption

Department of Computer Science and Engineering

Indian Institute of Technology Kanpur

Course: CS666 – Hardware Security of Internet of Things

Instructor: Assistant Professor Dr. Urbi Chatterjee

Date: 09 November 2025

CipherCore Group Members

Shubham Rawat
251110070

Major Sankalp Sharma
251110063

Kajal Kumari
251110040

Major Divakar Bajpai
251110029

Venkatesh Kumar
251110617

1 Abstract

1.1 Objective

- Implement a pipelined AES-128 Encryption Module using Verilog for the PYNQ board.

1.2 Key Improvements

- Converted the combinational key expansion into sequential, to reduce the critical path.
- Introduced pipeline stages for each encryption round, for higher throughput.

2 Design Overview

2.1 Base Reference

- The initial AES design was the non-pipelined version given on HelloIITK.
- It used a fully combinational key expansion and round execution.

2.2 Design Goals

- Reduce the critical path delay.
- Achieve one ciphertext output per clock cycle after pipeline fill.
- Maintain full AES-128 correctness.

2.3 Design Summary

- First KeyExpansion made sequential with three intra round registers (Inner pipeline), one each between Rotword, SubBytes and RCon operations. Followed by one register each between KeyExpansion rounds (Outer pipeline).
- Thereafter, introduced pipeline registers between each encryption round (Outer pipeline only).

3 Design Transition: From Non-Pipelined to Pipelined AES

3.1 Key Expansion Rework (Variation in KeyExpansion.v)

- **Before:** The non-pipelined KeyExpansion module computed all round keys combinationally, using nested generate loops and large fanout logic.

- **After:** Replaced with a multi-stage sequential FSM generating one round key per 3 clock cycles, reducing combinational delay and enabling pipelined launch. Round keys stored in an internal register array and updated incrementally.

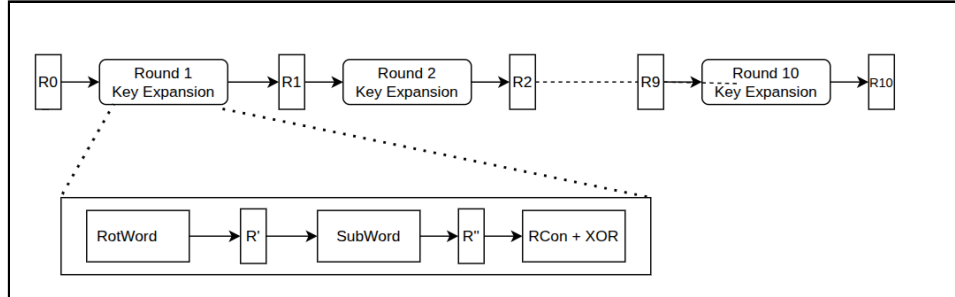


Figure 1: Block diagram after updated KeyExpansion

3.2 Data Path Modification (Variation in AESEncrypt.v)

- **Before:** Single AES round iteratively reused for all rounds, updating state in place.
- **After:** Created 10 dedicated AES round segments. Each operates on one block concurrently. Achieved continuous throughput once pipeline is full.

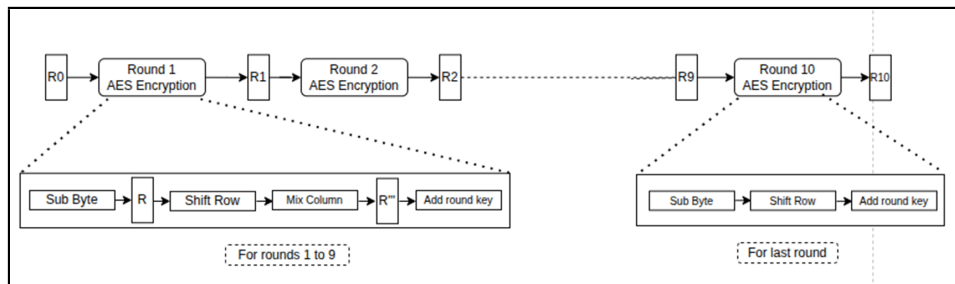


Figure 2: Block diagram after updated AESEncrypt

4 Architecture and Flow

4.1 Data Flow

- Plaintext and key loaded into registers.
- Key expansion runs sequentially to generate round keys.
- After 2 round of key expansion each pipeline stage applies SubBytes, ShiftRows, MixColumns, AddRoundKey (4 operations per clock cycle for first 9 rounds and 3 in last round of encryption).
- Final ciphertext emerges once all 10 stages are filled.

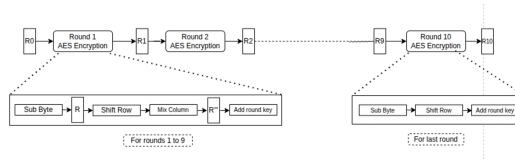


Figure 3: Enter Caption

4.2 Pipeline Timing

- Initial latency = 35 cycles.
- Thereafter, one output per clock.

5 Implementation Details

5.1 Module Hierarchy

- `AESEncrypt128_DUT` – The top-level module that instantiates both the `KeyExpansion` and the main `AESEncrypt` modules. However, during implementation on Vivado, a wrapper was used to package it for pin matching.
- `KeyExpansion` – A sequential, pipelined module responsible for generating all round keys from the initial key.
- `AESEncrypt` – The core encryption module implementing ten AES rounds.
- `SubBytes`, `ShiftRows`, `MixColumns`, `AddRoundKey`, and `SubTable` – Functional submodules used within each AES round to perform the standard AES transformations and S-box lookups. No changes carried out in these respective verilog files.

5.2 Hardware Setup

- Target board: PYNQ-Z2 (Zynq-7020 FPGA).
- Tools: Xilinx Vivado + Vitis.

6 Results and Evaluation

6.1 Functional Verification

- Sanity check. Setup and Hold time analysis after implementation

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 0.183 ns	Worst Hold Slack (WHS): 0.012 ns	Worst Pulse Width Slack (WPWS): 1.520 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 15413	Total Number of Endpoints: 15413	Total Number of Endpoints: 6604	
All user specified timing constraints are met.			

Figure 4: Design Timing Summary

- Output matched expected ciphertexts in simulation and on hardware.

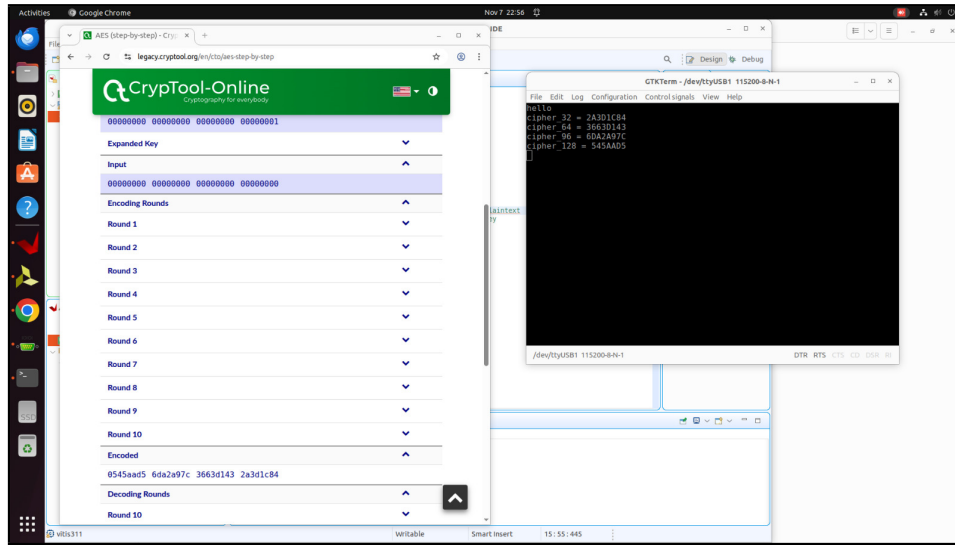


Figure 5: Output after successful implementation on PYNQ Z2

6.2 Performance Metrics

1. Slice Logic						
Site Type	Used	Fixed	Prohibited	Available	Util%	
Slice LUTs	4107	0	0	53200	7.72	
LUT as Logic	4047	0	0	53200	7.61	
LUT as Memory	60	0	0	17400	0.34	
LUT as Distributed RAM	0	0	0			
LUT as Shift Register	60	0	0			
Slice Registers	970	0	0	106400	0.91	
Register as Flip Flop	970	0	0	106400	0.91	
Register as Latch	0	0	0	106400	0.00	
F7 Muxes	794	0	0	26500	2.98	
F8 Muxes	153	0	0	13300	1.15	

(a) Non-Pipelined

1. Slice Logic						
Site Type	Used	Fixed	Prohibited	Available	Util%	
Slice LUTs	9110	0	0	53200	17.12	
LUT as Logic	9050	0	0	53200	17.01	
LUT as Memory	60	0	0	17400	0.34	
LUT as Distributed RAM	0	0	0			
LUT as Shift Register	60	0	0			
Slice Registers	6795	0	0	106400	6.39	
Register as Flip Flop	6539	0	0	106400	6.15	
Register as Latch	256	0	0	106400	0.24	
F7 Muxes	2688	0	0	26500	10.11	
F8 Muxes	1344	0	0	13300	10.11	

(b) Pipelined

Figure 6: Comparison of Area Utilization

Metric	Combinational	Pipelined (Proposed)	Unit
Operating Frequency	10	210	MHz
Throughput	0.11	26.8	Gbps
Initial Latency	11	35	cycles
LUT	4107	9110	numbers
Flip flop	970	6539	numbers
Area (Slices+FF)	5077	15649	–
Throughput $\times$ Area	558.47	420645.12	–

Table 1: Comparison of performance metrics between combinational and pipelined AES-128 designs.

7 Conclusion

- Pipelined AES-128 achieved a significant throughput improvement on the PYNQ board.
- Sequential key expansion reduced critical path and improved timing closure.

8 Enclosures

- Updated verilog code files (KeyExpansion1.v and AESEncrypt1.v) along with updated testbench (AESTestbench.v) enclosed and included in the zip file uploaded on HelloIITK.
- Vitis code (vitisCCode.c) also enclosed and included in the zip file, as mentioned in the instructions.
- Additionally, the updated AXI.v file (myipAXI.v) has also been enclosed and included in the zip file for recreating the project. 16 registers were initialized however only 15 were utilized.

References

1. Daemen, J., Rijmen, V. *The Design of Rijndael: AES Algorithm*. Springer, 2002.
2. Xilinx PYNQ Documentation. <https://pynq.io>
3. HelloIITK AES Reference Code.
4. Xilinx Vivado and Vitis User Guides.
5. chatGPT